

1. Protocolo

Dirección de archivo: C:\Users\marle\Documents\NetBeansProjects\EstructuraDatos\src\main\java\rational\

Tecnológico Nacional de México

Instituto Tecnológico de León, Campus 1

Carrera: Ingeniería en Sistemas Computacionales

Materia: Estructura de Datos (Grupo B)

Horario: Lunes y miércoles de 8:45 a 10:25 hrs. y
Viernes de 8:45 a 9:35 hrs.

Alumno: Marlene Inés Moreno Velázquez

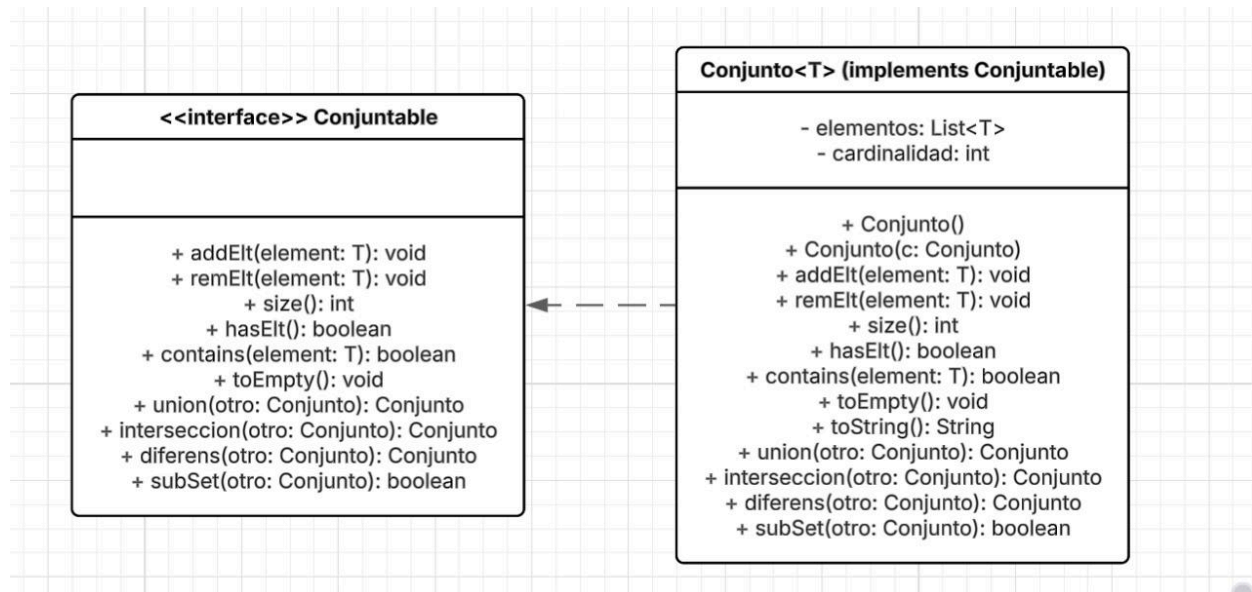
Número de lista: 12

Fecha: 11 de marzo de 2026.



Ejercicio #16: Conjunto

2. Diagrama UML



3. Clase encapsulada Conjunto

```
import java.util.ArrayList;

public class Conjunto implements Conjutable {
    private ArrayList<Integer> elementos;

    public Conjunto() {
        elementos = new ArrayList<>();
    }

    public void add(int e) {
        if (!elementos.contains(e)) elementos.add(e);
    }

    public void remove(int e) {
        elementos.remove((Integer) e);
    }

    public boolean contains(int e) {
        return elementos.contains(e);
    }

    public int size() {
        return elementos.size();
    }

    public Conjunto union(Conjunto c) {
        Conjunto res = new Conjunto();
        res.elementos.addAll(this.elementos);

        for (int e : c.elementos)
            if (!res.elementos.contains(e)) res.elementos.add(e);
        return res;
    }

    public Conjunto intersection(Conjunto c) {
        Conjunto res = new Conjunto();

        for (int e : elementos)
            if (c.elementos.contains(e)) res.add(e);
        return res;
    }

    public Conjunto difference(Conjunto c) {
        Conjunto res = new Conjunto();

        for (int e : elementos)
            if (!c.elementos.contains(e)) res.add(e);
        return res;
    }
}
```

```

    }

    @Override
    public String toString() {
        return elementos.toString();
    }
}

```

4. Algoritmo

1. meta():
 - Escribir(" El siguiente programa permite al usuario probar las diferentes prestaciones del programa Conjunto.");
2. data():
 - Conjunto a = new Conjunto();
 - Conjunto b = new Conjunto();
 - back = falso;
 - Escribir("Ingrese la cantidad deseada de elementos del conjunto A");
 - size = ?;
 - llamar a conjA(size);
 - Escribir("Ingrese la cantidad deseada de elementos del conjunto B");
 - size = ?;
 - llamar a conjB(size);
 - llamar a menuPrincipal();
- 2.2 conjA(int size):
 - para (i = 0 hasta i menor que size) empezar
 - Escribir("Ingrese un valor entero como elemento" + "\n(Valor por defecto = 0)");
 - elemento = ?;
 - a.add(elemento);
 - terminar para
- 2.3 conjB(int size):
 - para (i = 0 hasta i menor que size) empezar
 - Escribir("Ingrese un valor entero como elemento" + "\n(Valor por defecto = 0)");
 - elemento = ?;
 - b.add(elemento);
 - terminar para
3. menuOperaciones(int op):
 - segun op empezar
 - caso 0: Conjunto union = a.union(b);
 - llamar a resultado(union, op);
 - romper;
 - caso 1: Conjunto inter = a.intersection(b);
 - llamar a resultado(inter, op);

```

        romper;
    caso 2: Conjunto diff = a.difference(b);
        llamar a resultado(diff, op);
        romper;
    terminar segun

3.1 agregarElemento(int value, int wconj):
    segun wconj empezar
        caso 0: a.add(value); romper;
        caso 1: b.add(value); romper;
    terminar segun

3.2 eliminarElemento(int value, int wconj):
    segun wconj empezar
        caso 0: a.remove(value); romper;
        caso 1: b.remove(value); romper;
    terminar segun

3.3 encontrarElemento(int value, int wconj):
    segun wconj empezar
        caso 0: Escribir("Elemento " + value + " esta en el conjunto?\n"
            + a.contains(value)); romper;
        caso 1: Escribir("Elemento " + value + " esta en el conjunto?\n"
            + b.contains(value)); romper;
    terminar segun

4. resultado(Conjunto operacion, int op):
    segun op empezar
        caso 0: Escribir("A union con B = " + operacion);
            romper;
        caso 1: Escribir("A interseccion con B = " + operacion);
            romper;
        caso 2: Escribir("A diferencia con B = " + operacion);
            romper;
    terminar segun

5. main():
    llamar a meta();
    llamar a data();
    terminar mientras (!back) empezar
        llamar a menuPrincipal();
    terminar mientras

```

5. Algoritmo codificado en Java

```

import tools.EntryPrompt;

public class TestConjunto {
    EntryPrompt ep = new EntryPrompt();

```

```

Conjunto a = new Conjunto();
Conjunto b = new Conjunto();
boolean back = false;

void meta(){
    String mensaje = ""
        El siguiente programa permite al usuario probar las
        diferentes prestaciones del programa Conjunto.
        "";

    ep.outInfo(mensaje, "CONJUNTO - TEST PROGRAMA");
}

void data(){
    int size = ep.entryInt(1, "Conjunto A", "Ingrese la cantidad de "
        +"elementos de este conjunto."); conjA(size);
    size = ep.entryInt(1, "Conjunto B", "Ingrese la cantidad de "
        + "elementos de este conjunto."); conjB(size);

    menuPrincipal(); // Regresar a menu principal
}

void conjA(int size){
    int elemento;

    for(int i = 0; i < size; i++){
        elemento = ep.entryInt(0, "Agregar elemento",
            "Ingrese un valor entero como elemento. " +
            "\n(Valor por defecto = 0)");

        a.add(elemento);
    }
}

void conjB(int size) {
    int elemento;

    for (int i = 0; i < size; i++) {
        elemento = ep.entryInt(0, "Agregar elemento", "Ingrese un valor "
            + "entero como elemento. \n(Valor por defecto = 0)");

        b.add(elemento);
    }
}

void menuPrincipal(){
    String opciones1[] = {"Editar conjunto", "Realizar operacion",
        "Salir"};
    int opcion = ep.entryOption("Elija una opcion", "MENU PRINCIPAL",
        opciones1);
}

```



```

        conjB(size);}
    }}
    case 4 -> { switch(wconj){
        case 0 -> ep.outInfo("Tamaño de conjunto A = " +
            a.size(), "TAMAÑO DE CONJUNTO");
        case 1 -> ep.outInfo("Tamaño de conjunto B = " +
            b.size(), "TAMAÑO DE CONJUNTO");}}
    }
}

void agregarElemento(int value, int wconj){
    switch(wconj){
        case 0 -> a.add(value);
        case 1 -> b.add(value);
    }
}

void eliminarElemento(int value, int wconj){
    switch(wconj){
        case 0 -> a.remove(value);
        case 1 -> b.remove(value);
    }
}

void encontrarElemento(int value, int wconj){
    switch(wconj){
        case 0 -> ep.outInfo("Elemento " + value
            + " esta en el conjunto?\n" + a.contains(value),
            "Encontrar en Conjunto A");
        case 1 -> ep.outInfo("Elemento " + value
            + " esta en el conjunto?\n" + b.contains(value),
            "Encontrar en Conjunto B");
    }
}

void menuOperaciones(int op){
    switch(op){
        case 0 -> { Conjunto union = a.union(b);
            resultado(union, op);}
        case 1 -> { Conjunto inter = a.intersection(b);
            resultado(inter, op);}
        case 2 -> { Conjunto diff = a.difference(b);
            resultado(diff, op);}
    }
}

void resultado(Conjunto operacion, int op){
    switch(op){
        case 0 -> {ep.outPlain("A union con B = " + operacion,
            "RESULTADOS");}
    }
}

```

```

        case 1 -> {ep.outPlain("A interseccion con B = " + operacion,
                                "RESULTADOS");}
        case 2 -> {ep.outPlain("A diferencia con B = " + operacion,
                                "RESULTADOS");}
    }
}

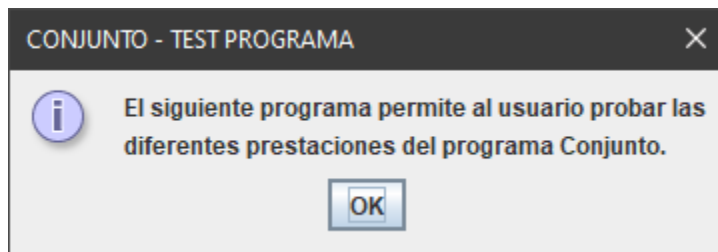
public static void main(String args[]){
    TestConjunto tc = new TestConjunto();

    tc.meta();
    tc.data();
    while(!tc.back) tc.menuPrincipal();
}
}

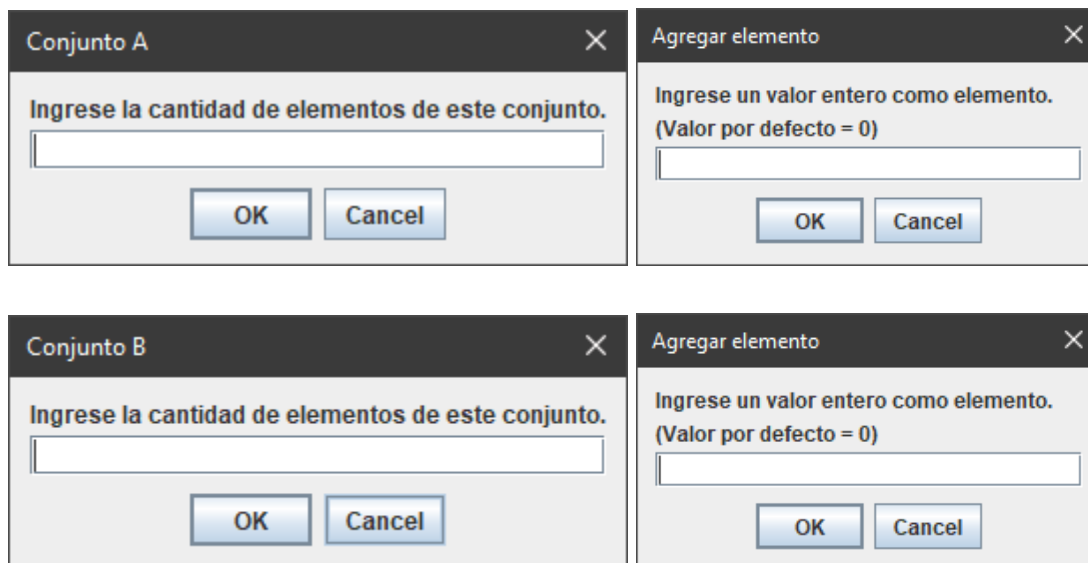
```

6. Resultados de corrida

Imagen demostrativa del Menú de inicio a continuación.



Imágenes demostrativas del programa solicitando el ingreso de datos al usuario a continuación.



Imágenes demostrativas del programa solicitando seleccionar una opción del menú principal, menú de edición y menú de operaciones.

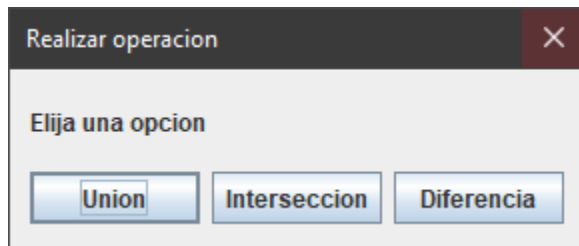
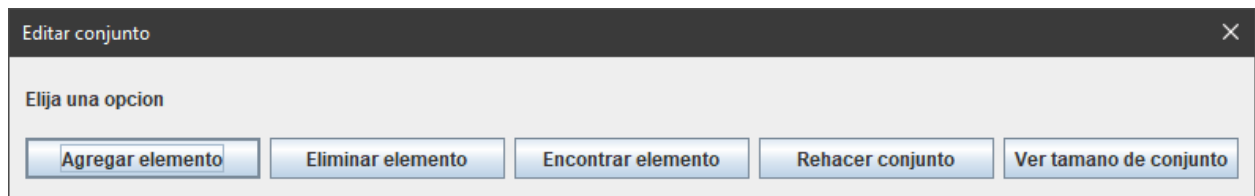
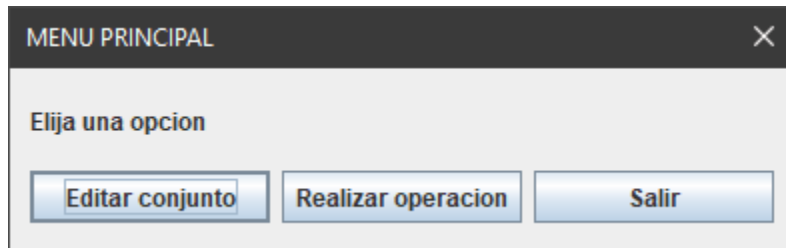


Imagen demostrativa del programa imprimiendo el resultado de la operación.

